

Homework 2

1. Create a new Vivado project. Write two models of a 4-to-1 multiplexer. One model should use an `if` statement and the other should use a `case` statement. Elaborate the two models and comment on the results.
2. Analyze `shiftreg.vhd` file for the functionality of included model. Identify all modes of operation of the 4-bit universal shift register component. On Figure 1 draw the routing necessary for possible implementation of such register.
3. Open the given `shiftreg.vhd` file in a Vivado project.
 - (a) Elaborate the circuit and analyze and explain how the two variables are implemented (as registers or wires).
 - (b) Move the output assignment “`(o_a, o_b, o_c, o_d) <= vreg;`” to inside the body of the “`rising_edge(i_clk)`” if statement and re-elaborate. Explain the change in the circuit.
4. On the page#3, analyze the given modules and do the following:
 - (a) Complete all the waveforms - **make delta cycles visible**
 - (b) Draw possible results from synthesis

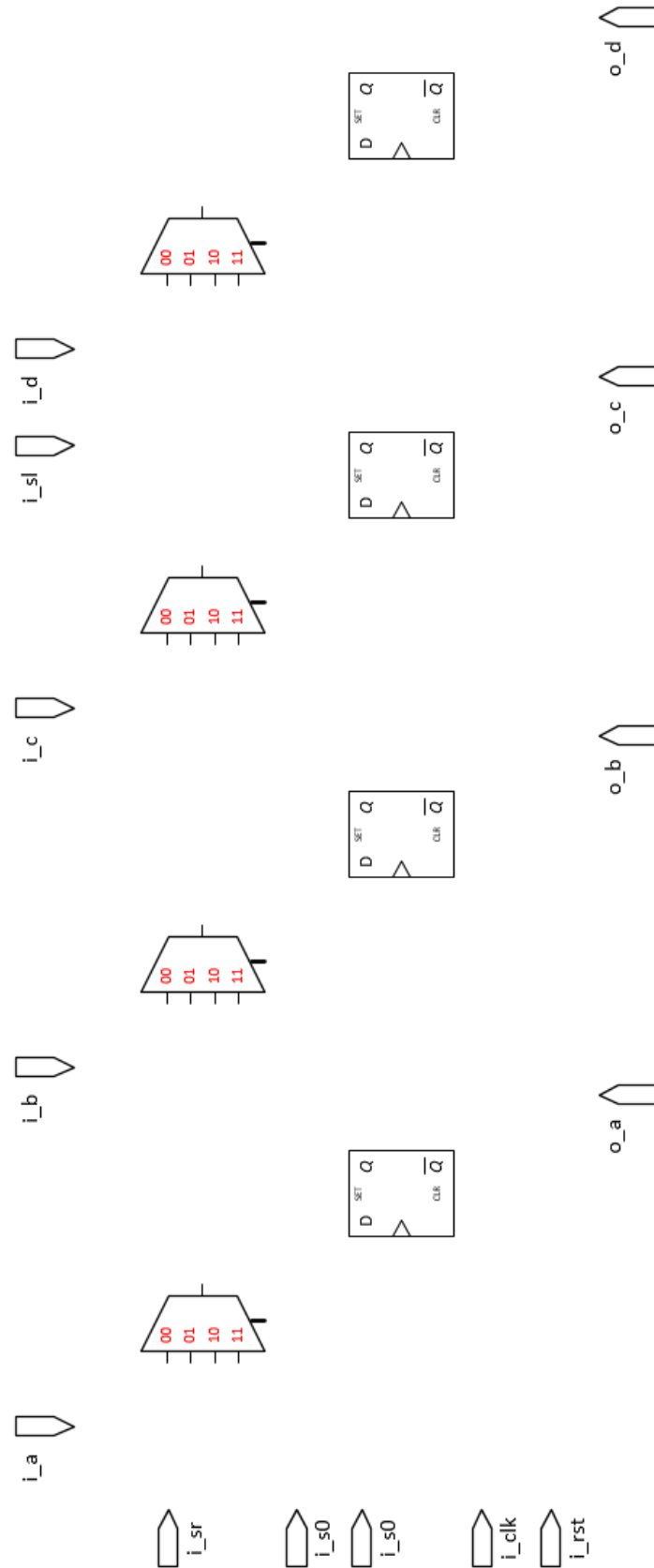


Figure 1: 4-bit universal shift register

Code	Waveform	Corresponding Circuit
<pre> library ieee; use ieee.std_logic_1164.all; entity reg is port (clk, a, b, c : in std_logic; dout : out std_logic); end entity reg; architecture cos of reg is signal t1 : std_logic; begin process (clk) variable t2 : std_logic; begin if (rising_edge(clk)) then t1 <= a nand b; t2 := t1 and c; dout <= t2; end if; end process; end architecture cos; </pre>		
<pre> library ieee; use ieee.std_logic_1164.all; entity reg is port (clk, a, b, c : in std_logic; dout : out std_logic); end entity reg; architecture cos of reg is begin process (clk) variable t1, t2: std_logic; begin if (rising_edge(clk)) then t1 := a nand b; t2 := t1 and c; dout <= t2; end if; end process; end architecture cos; </pre>		
<pre> library ieee; use ieee.std_logic_1164.all; entity reg is port (clk, a, b, c : in std_logic; dout : out std_logic); end entity reg; architecture cos of reg is signal t1,t2: std_logic; begin process (clk) begin if (rising_edge(clk)) then t1 <= a nand b; t2 <= t1 and c; dout <= t2; end if; end process; end architecture cos; </pre>		